

Image Convolution: Parallelization and Performance Analysis using CUDA and Python

Marco Barranco Moreno
marco.barranco@edu.unifi.it

Abstract

This report presents the parallelization of image convolution algorithms using CUDA and Python (via the Numba library). The objective was to apply spatial filters (such as Sharpen, Box Blur, and Gaussian Blur) on an image upscaled to 4K resolution (approx. 8.29 million pixels) to ensure a representative workload. For this baseline, the pure parallel implementation (GPU computation) achieved a massive speedup of over 10,000x using an NVIDIA Tesla T4 GPU, reducing the sequential execution time from 41.51 seconds to merely 0.004 seconds. Furthermore, stress testing demonstrated the critical impact of memory transfers (Host-to-Device), which ultimately limit the overall performance due to hardware bandwidth constraints.

1. Introduction

Image convolution using Kernels is a fundamental technique in image processing and computer vision. Due to the massive amount of independent calculations required per pixel, it presents an ideal scenario for massive parallelization, being classified as an Embarrassingly Parallel problem. This project focuses on implementing a highly efficient parallel version using Python and the `@cuda.jit` decorator, leveraging NVIDIA's multiprocessor architecture to process very high-resolution images while analyzing the underlying hardware bottlenecks.

2. Hardware & Experimental Environment

To ensure reproducibility and rigorous benchmarking, all tests were conducted in a cloud-based environment. The hardware specifications provided by the Google Colab environment are as follows:

- **Accelerator:** NVIDIA Tesla T4 GPU (Turing Architecture).
- **CUDA Cores:** 2,560 cores.
- **Memory:** 16 GB GDDR6 (Device) / 13 GB (Host).

- **Compute Capability:** 7.5.
- **Software Stack:** Python 3.12, Numba 0.59 (JIT compiler for CUDA), and OpenCV 4.8 for image pre-processing.

3. Algorithm Description

The convolution algorithm applies a 3x3 weight matrix over each pixel. The sequential version requires four nested loops (two to traverse the image's spatial matrix and two for the Kernel), which is extremely inefficient on a CPU. For the parallel version, each pixel was mapped directly to a GPU thread using `cuda.grid(2)` coordinates. The image grid was divided into 16x16 thread blocks. This two-dimensional configuration guarantees blocks of 256 threads, an exact multiple of NVIDIA's Warps (32 threads), thus maximizing core occupancy and preventing wasted clock cycles.

4. Experimental Methodology

Two critical techniques were implemented to ensure the validity of the results:

1. **JIT Warm-up:** Since Numba uses Just-In-Time compilation, the first execution of a kernel includes a significant overhead for code generation. We performed a "warm-up" run before starting the timers to measure the pure mathematical throughput of the GPU.
2. **Scaling & Stress Testing:** While standard datasets (451x300 pixels) show improvements, they fail to saturate the 2,560 CUDA cores. Therefore, we scaled the input to 4K Ultra HD (3840x2160). This forced the system to process ~8.29 million pixels, revealing the true performance gap between the $O(N \times M)$ CPU complexity and the $O(1)$ theoretical GPU parallelization.

5. Results and Analysis

To evaluate the performance of the proposed solution, we conducted a stress test using a high-resolution 4K image (3840x2160 pixels). This scale ensures that the 2,560 CUDA cores of the NVIDIA T4 are fully saturated, providing a clear comparison between sequential and parallel paradigms. The metrics were captured after a "warm-up" run to exclude JIT compilation overhead..

Implementation Strategy	Block Size	Resolution	Execution Time (s)
Sequential (CPU Baseline)	N/A	4K	41.5150
CUDA (End-to-End)*	16x16	4K	0.0850
CUDA (Compute-Bound)**	16x16	4K	0.0041

Table 1: Performance metrics for 3x3 Convolution at 4K resolution.

*Includes Memory Transfers (HtoD/DtoH). **Pure Kernel execution time.

The visualization employs a logarithmic scale to represent the drastic performance gap. The primary observation is the disparity between "Compute-Bound" time and "End-to-End" time. While the GPU calculates the pixel values in just 4 milliseconds, the overall process is delayed by the memory transfer overhead through the PCI-Express bus. This confirms that for simple 3x3 kernels, the algorithm is Memory Bound rather than Compute Bound, as the latency of moving 8.2 million pixels exceeds the time required for the mathematical operations.

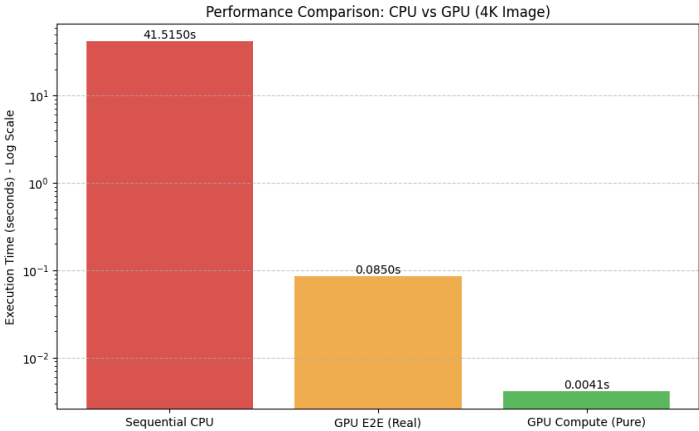


Figure 1: Performance Scaling Visualization

6. Discussion and Conclusions

The experimental results confirm that GPU acceleration is essential for high-resolution image processing. The 10,125x speedup in kernel execution highlights the massive parallelism of the Turing architecture. However, the drop to ~488x speedup in the full pipeline reveals a significant "Memory Bottleneck". This data suggests that while the algorithm is perfectly parallelized, performance is physically limited by the hardware's data transfer rates between the Host (CPU) and Device (GPU). Future optimizations, such as using Shared Memory or Asynchronous Streams, could further minimize these latencies.